

Exercise set 3

Shrinkage methods

Ridge regression and LASSO are called *shrinkage methods* since the regression coefficients are shrunk towards zero.

- OLS: Minimize $RSS(\beta) = \sum_{i=1}^n (y_i - \mathbf{x}_i' \beta)^2$
- Ridge regression: Minimize $RSS(\beta) + \lambda \sum_{k=1}^p \beta_k^2$
- LASSO: Minimize $RSS(\beta) + \lambda \sum_{k=1}^p |\beta_k|$

Task 1

Consider the dataset [College](#). You shall predict the number of applications, **Apps**. Restrict the dataset by

```
require(ISLR2)
xdata=subset(College,select=c("Apps","Accept","Top10perc","Expend"))
```

Use the help-function for [College](#) to get a description of the variables that you have chosen.

- a) Compare the estimated coefficients for OLS, ridge regression and LASSO in a regression of **Apps** on **Accept**, **Top10perc** and **Expend**. Try with different values for λ , $\lambda = 0.01, 0.1, 1, 10, 100, 1000$ and perhaps a few others. Ridge regression and LASSO can be fitted with the `glmnet`-function. As an example, to obtain the coefficients from Ridge regression use

```
library(glmnet)
beta_ridge=glmnet(as.matrix(xdata[,-1]),as.matrix(xdata[,1]),lambda=100,alpha=0)$beta
```

- b) Use the `cv.glmnet` function to find the optimal λ for both ridge regression and LASSO. Explain what the function is doing.
- c) Use the optimal λ 's and reestimate the ridge regression and the LASSO. Compare the coefficients with the OLS-estimates.
- d) Evaluate the three predictions by `testMSE` using the validation set approach.

Non-linear models

- In *polynomial regression* Y is regressed on $1, X, \dots, X^K$.
- In *piecewise constant regression* Y is regressed on indicator variables $I(X < c_1), I(c_1 \leq X < c_2), \dots, I(c_{K-1} \leq X < c_K), I(X \geq c_K)$.
- In the *basis function approach* Y is regressed on a number of transformations of X , $b_0(X), b_1(X), b_2(X), \dots, b_K(X)$.
- A *cubic spline basis* is defined by

$$\begin{aligned} & (b_0(x), b_1(x), b_2(x), b_3(x), b_4(x), \dots, b_{K+3}(x)) \\ & = (1, x, x^2, x^3, h(x, \xi_1), \dots, h(x, \xi_K)) \end{aligned}$$

where

$$h(x, \xi) = (x - \xi)_+^3 = \begin{cases} (x - \xi)^3 & \text{if } x > \xi \\ 0 & \text{otherwise} \end{cases}$$

- In *regression splines* Y is regressed on a cubic spline basis.

Task 2

Consider the data set

x	y
1	8.88
2	4.54
3	5.12
4	1.14
5	2.26
6	8.43
7	10.92
8	14.47

- Plot y against x .
- Fit a polynomial regression of order $K = 7$ to the data. Plot the predictions for $x = 1, \dots, 8$ on top of the data.
- Fit a regression spline using the `lm`-function with two knots, $\xi_1 = 3$ and $\xi_2 = 6$. Plot the predictions obtained on top of the data. Do you expect predictions from this model to be better or worse than the prediction in b) for a new x -value? Make sure that you understand what the code is doing.
- Use the R-function `bs` in the `splines`-package to produce the same predictions as in c).
- Investigate different placements of the knots and determine which of them are best using the leave-one-out cross-validation.

An extension of KNN

- In K nearest neighbors(KNN) we predict y_0 by

$$\hat{f}(x_0) = \frac{1}{K} \sum_{k=1}^K y_{(k)},$$

$y_{(k)}$ is the y -value corresponding to the x -value which is the k 'th closest to x_0 .

- An extension of KNN is to predict y_0 by

$$\hat{f}(x_0) = \frac{1}{n} \sum_{i=1}^n w_i y_i,$$

i.e., we predict y_0 by a weighted average of observed all y_i 's, $i = 1, 2, \dots, n$.

Task 3

- Two needed properties of the weights w_i , $i = 1, 2, \dots, n$ are that they should sum to one and that a large w_i should correspond to a small deviation between x_i and x_0 . Explain why we need these properties?
- One way, different from the way we have done it previously, to implement KNN in R is the following

```
weight=function(x0,x,K)
{
  n=length(x)
  w=matrix(0,n,1)
  dist=abs(x-x0)
  odist=order(dist)
```

```

w[odist[1:K]]=1/K
return(w)
}
knn2=function(x0,x,y,K=3)
{
  w=weight(x0,x,K=K)
  ypred=sum(w*y)
  return(ypred)
}

```

Make sure that you understand why this give the same result as

```

knn1=function(x0,x,y,K=3)
{
  dist=abs(x-x0)
  odist=order(dist)
  y0=mean(y[odist[1:K]])
  return(y0)
}

```

Experiment with a small dataset, e.g.

```

set.seed(12)
x=seq(-50,50,length=100)
y=1-0.05*x-0.02*x^2+0.003*x^3+50*rnorm(100)

```

to see and understand the equality of knn1 and knn2.

- c) If we change the weight function to

$$w_i = K(x_i - x_0) = \frac{\phi\left(\frac{x_i - x_0}{h}\right)}{\sum_{i=1}^n \phi\left(\frac{x_i - x_0}{h}\right)}$$

where ϕ is the standard normal density we get the Nadaraya-Watson estimator. Modify the function knn2 above so it becomes the Nadaraya-Watson estimator. Apply it to the data simulated in b). Plot the predictions on top of the data. Try with a few different values of h . (Hint: the weights can be computed by `weight=function(x0,x,h=0.2) dnorm((x-x0)/h)/sum(dnorm((x-x0)/h))` in R).

- d) What is the meaning of h in c)?